2026 IEEE INTERNATIONAL WORKSHOP ON MACHINE LEARNING FOR SIGNAL PROCESSING, SEP. 28– OCT. 1, 2026, ATLANTA, USA

DYLNC: DYNAMICS-AWARE LATENT NETWORK COMPONENT DECOMPOSITION FOR TEMPORAL NETWORKS

Efe Karakoca

Dept. of Electrical and Electronics Eng.
Boğaziçi University
Istanbul, Türkiye
efe.karakoca@std.bogazici.edu.tr

Selin Aviyente

Dept. of Electrical and Computer Eng.
Michigan State University
East Lansing, MI, USA
aviyente@msu.edu

ABSTRACT

Temporal networks often evolve through recurring structural patterns whose activations change over time. A key challenge is to recover these connectivity patterns while also explaining how their activations progress across time. In this paper, we propose a dynamics-aware latent network decomposition that represents each graph snapshot as a linear combination of latent network components with time-varying coefficients. Unlike decomposition-only approaches that treat temporal coefficients as free variables or impose only smoothness priors, the proposed formulation jointly learns the latent components, coefficient trajectories, and a low-dimensional progression rule. To capture the deviations not explained by autonomous evolution, we include a learned exogenous forcing term and impose multi-step consistency for stable long-term dynamics. Experiments on synthetic and real temporal network data show that the proposed model recovers interpretable latent network components while capturing temporal evolution.

***Index Terms*—** Temporal networks, latent network components, network reduction, non-autonomous dynamics

1. INTRODUCTION

Temporal networks arise in many applications [1], including social contact systems [2] and financial correlation networks [3] and often exhibit recurring structural patterns such as repeated contact groups or dominant interaction regimes. In order to understand the underlying dynamics of temporal networks, one needs to extract a small number of latent network components and the corresponding temporal trajectory; thus, disentangling the node connectivity patterns from the temporal dynamics.

A common approach to analyzing temporal networks is to stack time-indexed adjacency matrices into a tensor and apply low-rank decomposition. CP and nonnegative tensor factorization methods decompose temporal adjacency tensors into node and temporal activity factors [4, 5], and related tensor methods have been used to identify dynamic functional brain network states from EEG [6, 7]. More expressive variants such as block-term decomposition (BTD) extend beyond rank-one community patterns [8], and tensor-based topology inference has been used for directed-graph identification and dynamic network tracking [9]. However, the resulting factors are tensor-algebraic components rather than graph-constrained adjacency templates, so the spatial factors may not correspond to interpretable graph structures. This limitation has motivated latent graph decomposition approaches that explicitly represent each snapshot as a linear combination of latent network components [10].

Latent graph decomposition alone, however, does not explain how the activations of the latent graphs evolve over time. In most existing methods, the temporal coefficients are treated as independent variables, so the model reveals recurring patterns without discovering the dynamics that govern the transitions between them. Continuous-time and graph-ODE methods [11, 12] address temporal evolution, but they focus on node-state or edge-embedding dynamics rather than on the activation coefficients of interpretable latent graphs. Graph Kalman filtering methods also adopt a state-space view of graph dynamics [13]. However, as in standard Kalman filtering, their formulation is naturally organized around recursive one-step prediction and measurement-based correction, rather than directly learning dynamics through a multi-horizon rollout objective. In addition, the learned transition map can absorb both autonomous and input-driven effects, so it does not explicitly separate interpretable latent-graph activations into autonomous progression and exogenous forcing. On the other hand, real temporal networks often require the separation of natural latent progression from time-dependent external deviations.

In this paper, we propose a dynamics-aware latent network component decomposition (DyLNC) for temporal networks. Each snapshot is represented as a linear combination of sparse latent network components, and the activation co-

979-8-3195-0884-3/26/$31.00 ©2026 IEEE

efficients are constrained to follow a non-autonomous progression with a learned exogenous forcing term. The model preserves the interpretability of latent graph decompositions while adding a dynamical mechanism for explaining how the activation of the latent network components evolve.

2. BACKGROUND

2.1. Graph Notation

Let $\mathcal{G} = \{G_t\}_{t=1}^T$ be a temporal sequence of weighted, undirected, and unsigned graphs on a fixed node set $\mathcal{V} = \{1, \dots, N\}$. The graph at time t is represented by an adjacency matrix $\mathbf{W}_t \in \mathbb{R}^{N\times N}$, where $(\mathbf{W}_t)_{ij}$ is the edge weight between nodes i and j. Since the graphs are undirected and have no self-loops, $\mathbf{W}_t = \mathbf{W}_t^\top$ and $\mathrm{diag}(\mathbf{W}_t) = 0$. The degree matrix is $\mathbf{D}_t = \mathrm{diag}(\mathbf{W}_t\mathbf{1})$, and the graph Laplacian is $\mathbf{L}_t = \mathbf{D}_t - \mathbf{W}_t$. For nonnegative edge weights, $\mathbf{L}_t$ is symmetric positive semidefinite and satisfies $\mathbf{L}_t\mathbf{1} = 0$.

Since the graphs are undirected, each snapshot can be represented by its strictly upper-triangular edge vector $\mathbf{y}_t = \mathrm{vec}_{\mathrm{ut}}(\mathbf{W}_t) \in \mathbb{R}^E$, where $E = N(N-1)/2$. Stacking all snapshots gives $\mathbf{Y} = [\mathbf{y}_1, \dots, \mathbf{y}_T] \in \mathbb{R}^{E\times T}$.

2.2. Latent Network Decomposition

Assume that the temporal graph sequence can be represented using R latent network components, where each $\mathbf{a}_r$ corresponds to a symmetric zero-diagonal graph pattern. Let $\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_R] \in \mathbb{R}^{E\times R}$ be the matrix of vectorized latent network components. The time-varying activation coefficients can be represented by $\mathbf{C} = [\mathbf{c}_1^\top, \dots, \mathbf{c}_T^\top]^\top \in \mathbb{R}^{T\times R}$, where $\mathbf{c}_t \in \mathbb{R}^{1\times R}$ is the latent activation vector at time t. The latent network decomposition is then given by $\mathbf{y}_t \approx \mathbf{A}\mathbf{c}_t^\top = \sum_{r=1}^R c_{t,r}\mathbf{a}_r$, $\mathbf{Y} \approx \mathbf{A}\mathbf{C}^\top$.

To improve identifiability and interpretability, each of the unsigned latent network components (LNCs) is constrained to the capped simplex

$$\mathcal{A} = \left\{\mathbf{a} \in \mathbb{R}^E : 0 \le a_i \le u_A,\ \mathbf{1}^\top\mathbf{a} = m_A\right\}, \quad \mathbf{a}_r \in \mathcal{A}, \tag{1}$$

where m_A fixes the ℓ_1-norm of each component and removes the scaling ambiguity between $\mathbf{A}$ and $\mathbf{C}$, while u_A limits the maximum weight assigned to any single edge. Consequently, u_A controls the minimum support size and prevents degenerate LNC composed of only a few edges.

2.3. Autonomous and Non-Autonomous Progression

To model temporal progression in the latent coefficient space, an autonomous linear model can be written in continuous/discrete time as $\mathrm{d}\mathbf{c}(\tau)/\mathrm{d}\tau = \mathbf{c}(\tau)\mathbf{\Theta}$, and $\mathbf{c}_{t+1} = \mathbf{c}_t\mathbf{F}$, respectively, where $\mathbf{F} = \exp(\Delta\mathbf{\Theta})$. Here, $\mathbf{\Theta} \in \mathbb{R}^{R\times R}$ is the continuous-time system matrix, Δ is the sampling interval, and $\mathbf{F} \in \mathbb{R}^{R\times R}$ is the corresponding discrete-time latent progression matrix.

A purely autonomous model assumes that the next latent state is determined only by the current state. Since temporal networks may be driven by time-dependent external forces, we use the non-autonomous model [14] $\mathbf{c}_{t+1} \approx \mathbf{c}_t\mathbf{F} + \mathbf{B}_t$, where $\mathbf{B}_t \in \mathbb{R}^{1\times R}$. Here, $\mathbf{F}$ captures the shared autonomous progression, while $\mathbf{B}_t$ is a learned exogenous forcing term. For a rollout horizon k, this model gives $\hat{\mathbf{c}}_{t+k|t} = \mathbf{c}_t\mathbf{F}^k + \sum_{j=0}^{k-1}\mathbf{B}_{t+j}\mathbf{F}^{k-1-j}$, $\mathbf{E}_{t,k} = \hat{\mathbf{c}}_{t+k|t} - \mathbf{c}_{t+k}$. The residual $\mathbf{E}_{t,k}$ measures the multi-step consistency and will be used to ensure that the learned progression remains consistent over multiple temporal horizons.

3. PROBLEM FORMULATION AND PROPOSED METHOD

Given a temporal network $\mathcal{G} = \{G_1, G_2, \dots, G_T\}$ with corresponding adjacency matrices $\{\mathbf{W}_t\}_{t=1}^T$, our goal is to jointly learn the LNCs $\mathbf{A}$, the activation coefficients $\mathbf{C}$, the autonomous latent progression matrix $\mathbf{F}$, and the exogenous forcing sequence $\mathbf{B} = [\mathbf{B}_1^\top, \dots, \mathbf{B}_{T-1}^\top]^\top \in \mathbb{R}^{(T-1)\times R}$.

3.1. Objective Function

Using the notation introduced in Section 2, the proposed optimization problem is

$$\begin{aligned}\min_{\mathbf{A},\mathbf{C},\mathbf{F},\mathbf{B}} \quad & \frac{1}{2}\|\mathbf{Y} - \mathbf{A}\mathbf{C}^\top\|_F^2 + J_{\mathrm{dyn}}(\mathbf{C},\mathbf{F},\mathbf{B}) + \mathcal{R}(\mathbf{A},\mathbf{C},\mathbf{F},\mathbf{B}) \\ \text{s.t.} \quad & \mathbf{a}_r \in \mathcal{A}, \quad r = 1,\dots,R, \qquad \mathbf{C} \ge 0.\end{aligned} \tag{2}$$

The first term in (2) is the reconstruction loss. To make the coefficient trajectory dynamically constrained, we penalize the multi-step residuals, $\mathbf{E}_{t,k}$ for a small set of rollout horizons $\mathcal{K}$ as:

$$J_{\mathrm{dyn}}(\mathbf{C},\mathbf{F},\mathbf{B}) = \sum_{k\in\mathcal{K}} \frac{\nu_k}{2(T-k)} \sum_{t=1}^{T-k} \|\mathbf{E}_{t,k}\|_2^2, \tag{3}$$

where ν_k controls the importance of horizon k. This term prevents the progression model from only fitting one-step transitions and encourages $\mathbf{F}$ and $\mathbf{B}$ to generate coefficient trajectories that remain accurate over multiple rollout lengths.

We further regularize the learned LNC and latent trajectories with

$$\begin{aligned}\mathcal{R} = \lambda_* \sum_{r=1}^R \|\mathcal{M}(\mathbf{a}_r)\|_* + \beta \sum_{r<s} \mathbf{a}_r^\top\mathbf{a}_s + \frac{\mu_C}{2(T-2)}\|\mathbf{D}_2\mathbf{C}\|_F^2 \\ + \frac{\lambda_F}{2}\|\mathbf{F}\|_F^2 + \frac{\lambda_B}{2}\|\mathbf{B}\|_F^2,\end{aligned} \tag{4}$$

where $\mathcal{M}(\mathbf{a}_r)$ denotes the symmetric zero-diagonal adjacency matrix reconstructed from the upper-triangular edge vector $\mathbf{a}_r$. The nuclear norm penalty $\sum_{r=1}^{R} \|\mathcal{M}(\mathbf{a}_r)\|_*$ encourages each LNC to have coherent low-dimensional graph structure, while the overlap penalty $\sum_{1\le r<s\le R} \mathbf{a}_r^\top \mathbf{a}_s$ discourages multiple LNCs from explaining the same set of edges. The second-order smoothness term $\|\mathbf{D}_2\mathbf{C}\|_F^2$ uses the finite-difference matrix $\mathbf{D}_2 \in \mathbb{R}^{(T-2)\times T}$, such that $(\mathbf{D}_2\mathbf{C})_{t,:} = \mathbf{c}_{t+2} - 2\mathbf{c}_{t+1} + \mathbf{c}_t$, which discourages high-frequency oscillations in the coefficient trajectory. Finally, $\mathbf{B}$ and $\mathbf{F}$ are regularized to avoid the external force to dominate in explaining the dynamics.

3.2. Block-Coordinate Optimization

We solve (2) using the block-coordinate procedure in Algorithm 1. Our objective function is a nonsmooth nonconvex block-variable problem, which conforms to the inexact block successive convex approximation framework of Yang et al. [15] and satisfies the assumptions mentioned in that framework. Thus, every limit point of the generated sequence is stationary. The implementation is available online. [1]

4. EXPERIMENTAL SETUP

4.1. Simulated Temporal Networks

We first evaluate the proposed model on synthetic temporal networks with known ground-truth LNCs, coefficient trajectories, and dynamics. We then compare the proposed method with respect to two tensor-decomposition baselines: (1) block term decomposition (BTD); (2) Symmetric nonnegative rank-$(L, L, 1)$ BTD variant following the Block Component Analysis (BCA) framework [16].

4.1.1. Synthetic Data Generation

We generate an undirected temporal network with $N = 60$ nodes, $T = 40$ snapshots, and $R = 2$ latent LNCs, $\mathbf{A}_\star = [\mathbf{a}_1, \mathbf{a}_2] \in \mathbb{R}_+^{E\times 2}$, where $E = N(N-1)/2$. The LNCs are generated from prescribed block supports. The first LNC $\mathbf{a}_1$ contains five dominant diagonal within-community blocks together with two weak off-diagonal interaction blocks, whereas the second LNC $\mathbf{a}_2$ contains another two diagonal blocks and five off-diagonal cross-community blocks. Within each block, edges are selected randomly using Bernoulli sampling and assigned weights from a uniform distribution. The resulting matrices are symmetrized, their diagonals are removed, and the upper-triangular LNC vectors are projected onto the capped simplex. The temporal coefficients are generated according to $\mathbf{c}_{t+1} = \mathbf{c}_t\mathbf{F} + \mathbf{b}_t$, where $\mathbf{F}$ is a damped rotational transition matrix. The forcing sequence is obtained from Gaussian-smoothed Piecewise Cubic Hermite Interpolating Polynomial (PCHIP) interpolation of specified anchor points, followed by an affine transformation and additive noise. The clean edge sequence is formed as $\mathbf{Y}_{\text{clean}} = \mathbf{A}_\star\mathbf{C}^\top$, and the observed sequence is obtained by adding Gaussian noise followed by nonnegative clipping.

[1] Code: `https://github.com/Efe6402/research_mlsp`

Algorithm 1 BCD-based solver for DyLNC

Require: Temporal edge matrix $\mathbf{Y}$, number of LNCs R, horizons $\mathcal{K}$, weights $\{\nu_k\}_{k\in\mathcal{K}}$
Ensure: $\mathbf{A}$, $\mathbf{C}$, $\mathbf{F}$, and $\mathbf{B}$

Initialize $\mathbf{A}, \mathbf{C}$ by NMF: $\mathbf{Y} \approx \mathbf{A}\mathbf{C}^\top$; project $\mathbf{a}_r \leftarrow \Pi_{\mathcal{A}}(\mathbf{a}_r)$.
Initialize $\mathbf{F}$ by ridge regression from $\mathbf{c}_{t+1} \approx \mathbf{c}_t\mathbf{F}$; set $\mathbf{b}_t \approx \mathbf{c}_{t+1} - \mathbf{c}_t\mathbf{F}$.
for $i = 1, \dots, I_{\text{outer}}$ **do**
  LNC update (A): for $r = 1, \dots, R$, form $\mathbf{R}_r = \mathbf{Y} - \sum_{s\neq r} \mathbf{a}_s\mathbf{c}_s^\top$, $q_r = \|\mathbf{c}_r\|_2^2$, $\mathbf{p}_r = \mathbf{R}_r\mathbf{c}_r$, $\mathbf{b}_r = \sum_{s\neq r} \mathbf{a}_s$.
  Set $\mathbf{g}_r = q_r\mathbf{a}_r - \mathbf{p}_r + \beta\mathbf{b}_r$, $\mathbf{v}_r = \mathbf{a}_r - \alpha\mathbf{g}_r$, and update $\mathbf{a}_r$ by inner ADMM:
$$\min_{\mathbf{a}\in\mathcal{A}} \frac{1}{2\alpha}\|\mathbf{a} - \mathbf{v}_r\|_2^2 + \lambda_\star\|\mathcal{M}(\mathbf{a})\|_*.$$
  In ADMM, split $\mathbf{X} = \mathcal{M}(\mathbf{a})$; use singular-value thresholding (SVT) for $\mathbf{X}$ and capped-simplex projection for $\mathbf{a}$. Choose α by Armijo backtracking.
  Coefficient update (C): compute $\mathbf{G}_C = \mathbf{G}_{C,\text{rec}} + \mathbf{G}_{C,\text{dyn}} + \mathbf{G}_{C,\text{smth}}$, where $\mathbf{G}_{C,\text{rec}}, \mathbf{G}_{C,\text{dyn}}, \mathbf{G}_{C,\text{smth}}$ are the gradients of the reconstruction (2), dynamics (3), and smoothness terms (4)
  Update $\mathbf{C} \leftarrow [\mathbf{C} - \eta_C\mathbf{G}_C]_+$ with Armijo backtracking.
  Forcing update (B): for $k \in \mathcal{K}$, set $\mathbf{C}_{-k} = \mathbf{C}(1 : T-k, :)$, $\mathbf{C}_{+k} = \mathbf{C}(1+k : T, :)$, $\mathbf{R}_k = \mathbf{C}_{+k} - \mathbf{C}_{-k}\mathbf{F}^k$.
  Define $\mathcal{L}_k(\mathbf{B})(t,:) = \sum_{j=0}^{k-1} \mathbf{B}(t+j,:)\mathbf{F}^{k-1-j}$. Let $\text{vec}(\mathcal{L}_k(\mathbf{B})) = \mathbf{K}_k\mathbf{b}$, $\mathbf{b} = \text{vec}(\mathbf{B})$, $\mathbf{r}_k = \text{vec}(\mathbf{R}_k)$, with
$$\mathbf{K}_k[t+(q-1)m_k,\ s+(p-1)n] = \begin{cases} [\mathbf{F}^{k-1-(s-t)}]_{p,q}, & 0 \le s-t \le k-1, \\ 0, & \text{otherwise}, \end{cases}$$
  where $n = T-1$ and $m_k = T-k$.
  With $\alpha_k = \nu_k/(T-k)$, solve
$$\left(\sum_{k\in\mathcal{K}} \alpha_k\mathbf{K}_k^\top\mathbf{K}_k + \lambda_B\mathbf{I}\right)\mathbf{b} = \sum_{k\in\mathcal{K}} \alpha_k\mathbf{K}_k^\top\mathbf{r}_k,$$
  and reshape $\mathbf{b}$ to obtain $\mathbf{B}$.
  Autonomous Progression update (F): for $k \in \mathcal{K}$, define
$$\mathbf{E}_k = \mathbf{C}_{-k}\mathbf{F}^k + \sum_{j=0}^{k-1} \mathbf{B}_{k,j}\mathbf{F}^{k-1-j} - \mathbf{C}_{+k}, \quad \mathbf{B}_{k,j} = \mathbf{B}(1+j : T-k+j, :).$$
  Use
$$\Gamma_m(\mathbf{X}, \mathbf{E}; \mathbf{F}) = \sum_{p=0}^{m-1} (\mathbf{F}^p)^\top\mathbf{X}^\top\mathbf{E}(\mathbf{F}^{m-1-p})^\top, \quad \Gamma_0 = 0,$$
  and compute
$$\mathbf{G}_F = \sum_{k\in\mathcal{K}} \alpha_k \left[\Gamma_k(\mathbf{C}_{-k}, \mathbf{E}_k; \mathbf{F}) + \sum_{j=0}^{k-2} \Gamma_{k-1-j}(\mathbf{B}_{k,j}, \mathbf{E}_k; \mathbf{F})\right] + \lambda_F\mathbf{F}.$$
  Update $\mathbf{F} \leftarrow \mathbf{F} - \eta_F\mathbf{G}_F$ with Armijo backtracking.
  if relative objective decrease is below tolerance **then**
    break
  end if
end for

4.1.2. Hyperparameter Selection

The hyperparameters are selected to balance three criteria: accurate temporal adjacency reconstruction, separation of the learned LNCs, and consistency of the coefficient rollout generated from the learned progression rule. Furthermore, the parameters m_A and u_A are chosen to satisfy the condition $m_A \le Eu_A$ for the $\mathcal{A}$ capped simplex set feasibility. We select $m_A = 1$, $u_A = 9.5 \times 10^{-3}$, $\lambda_* \in [2, 5] \times 10^{-2}$, $\beta \in \left[5 \times 10^{-2}, 5 \times 10^{-1}\right]$, $\mu_C \in [1, 2] \times 10^{-2}$, $\lambda_B \in [1, 5] \times$

10^{-2}, $\eta_F \in [0.45, 0.50]$, and use $\mathcal{K} = \{1, 2, 3, 5, 8, 13, 18\}$.

For BTD methods, since the representational capacity depends on the prescribed block rank L, we select L from the singular-value energy profile of the ground-truth LNC to provide a comparable evaluation setting. For each LNC $\mathbf{a}_r$, with singular values $\{\boldsymbol{\sigma}(\mathcal{M}(\mathbf{a}_r))\}_i$, we define $L_r^{(q)} = \min\left\{k : \frac{\sum_{i=1}^{k}\sigma_i^2}{\sum_i \sigma_i^2} \geq q\right\}$, and $L^{(q)} = \left\lceil \frac{1}{R}\sum_{r=1}^{R} L_r^{(q)} \right\rceil$. where q denotes the percentile recovered energy. This procedure gives the average energy-rank choices of $L^{(0.90)} = 16$ and $L^{(0.99)} = 29$.

4.1.3. *Evaluation*

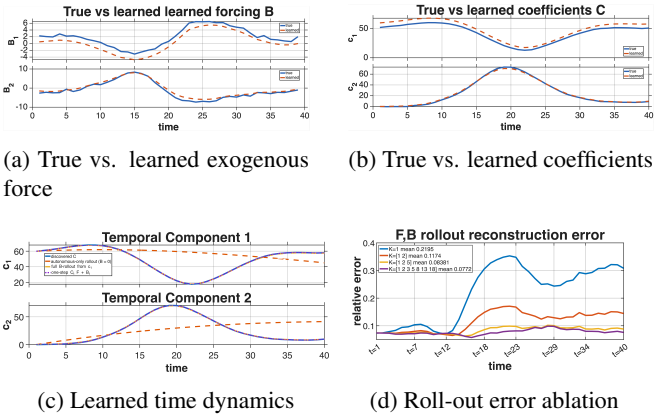
For LNC-level evaluation, LNCs are first normalized by their Frobenius norm after reshaping into adjacency matrices and then permutation-matched. We report the relative LNC recovery error $\|\mathbf{a} - \hat{\mathbf{a}}\|_2^2 / \|\mathbf{a}\|_2^2$, the nuclear norm of the normalized adjacency matrix, the within/across-LNC cosine distance, and the Hoyer sparsity $(\sqrt{E} - \|\mathbf{a}\|_1/\|\mathbf{a}\|_2)/(\sqrt{E} - 1)$.

The results in Table 1 show that DyLNC accurately recovers the generated LNCs while preserving their low-rank structure, sparsity, and component separation. The learned LNCs exhibit nuclear-norm and Hoyer-sparsity values close to the true templates, low within-LNC cosine distance, and high cross-LNC separation. With the near-oracle rank choice $L = L^{(0.99)}$, the BTD baselines achieve comparable reconstruction accuracy, confirming sufficient capacity. However, for the more compressed energy-based choice of $L = L^{(0.90)}$, the errors increase noticeably, highlighting the sensitivity of BTD sensitivity to the prescribed block rank, whereas DyLNC does not require explicit specification of this internal rank.

Beyond reconstruction, DyLNC recovers the latent progression; the rollout generated by the learned $\mathbf{F}$ and $\mathbf{B}$ closely follows the recovered coefficient trajectory, while the autonomous rollout with $\mathbf{B} = 0$ fails to capture the non-autonomous deviations, as shown in Fig. 1c. The rollout-horizon ablation further shows that short horizons such as $K = \{1\}$ and $K = \{1, 2\}$ accumulate error over time, whereas $K = \{1, 2, 3, 5, 8, 13, 18\}$ maintains consistently low rollout error, confirming the importance of multi-step consistency in Fig. 1d.

4.2. Real-Data Experiments

We evaluate DyLNC on two real temporal network datasets with different underlying mechanisms: a face-to-face contact network and a financial correlation network. For real-data experiments, we select the BTD block L from the observed temporal graph sequence. For each snapshot edge vector $\mathbf{y}_t = \mathbf{Y}_{:,t}$, we compute the singular-energy rank $k_t^{(q)}$ of the reshaped adjacency matrix $\mathcal{M}(\mathbf{y}_t)$, defined as the smallest k satisfying $\sum_{i=1}^{k}\sigma_i^2(\mathcal{M}(\mathbf{y}_t))/\sum_i \sigma_i^2(\mathcal{M}(\mathbf{y}_t)) \geq q$. Since each snapshot is a mixture of R latent components, we set the per-component BTD rank to $L = \lceil \max_t k_t^{(q)}/R \rceil$, so that the total BTD capacity satisfies $RL \geq \max_t k_t^{(q)}$ for $q = 0.90$ and $q = 0.95$.



(a) True vs. learned exogenous force

(b) True vs. learned coefficients

(c) Learned time dynamics

(d) Roll-out error ablation

Fig. 1: DyLNC dynamic recovery and ablations. (a)–(b) compare learned and true dynamics. (c) shows autonomous-only ablation. (d) shows the ablation of multi-step consistency.

4.2.1. *Primary School Contact Network*

For the first real-data experiment, we use the SocioPatterns Primary School contact dataset [17], which records face-to-face proximity interactions among students and teachers during a school day from 8:30 a.m. to 4:30 p.m. with a lunch break from 12:00 p.m. to 2:00 p.m. Motivated by the presence of these two time periods, we set $R = 2$ for this dataset. The raw sensor contacts are aggregated over approximately 13-minute intervals, resulting in a temporal network tensor $\mathcal{X} \in \mathbb{R}^{242\times242\times40}$.

As reported in Table 2, DyLNC achieves comparable or lower reconstruction error than the BTD baselines while maintaining high template separability. In Fig. 2a, the learned LNCs reveal two interpretable modes: localized within-class contacts and a more diffuse interaction pattern, consistent with classroom and break-time activity, respectively. The rollout in Fig. 2c closely follows the discovered coefficient trajectories, while Fig. 2b shows increased force contribution near the beginning and end of lunch break, indicating schedule-driven deviations from autonomous progression.

4.2.2. *S&P 500 Financial Correlation Network*

For the second real-data experiment, we use the S&P 500 stock-price dataset from [18]. Daily log-returns are computed for $p = 100$ stocks, and Pearson correlation matrices are computed using a 60-trading-day window with a 10-day step size. To obtain unsigned graph snapshots, we take the entry-wise absolute value of each correlation matrix, remove the diagonal, and retain the largest off-diagonal entries corresponding to a target density of 0.15. Sector information is used to reorder the nodes for visualization and interpretation.

Table 1: LNC Recovery and Reconstruction Metrics for Synthetic Data

Method	L	Temp Recon. Err.	Nuclear Norm	Rel. MSE to LNC	Within-LNC Cos. Dist.	Cross-LNC Cos. Dist.	Hoyer
True generated	–	0.000000	4.6777 ± 0.3532	0.0000 ± 0.0000	0.0000 ± 0.0000	0.8229 ± 0.0000	0.7735 ± 0.0010
DyLNC	–	**0.039488**	4.6890 ± 0.3544	$\mathbf{0.0008 \pm 0.0010}$	$\mathbf{0.0004 \pm 0.0005}$	$\mathbf{0.8262 \pm 0.0000}$	$\mathbf{0.7555 \pm 0.0230}$
Classical BTD	16	0.176447	4.5704 ± 0.2629	0.0473 ± 0.0071	0.0236 ± 0.0036	$\mathbf{0.8692 \pm 0.0000}$	0.7100 ± 0.0008
BCA BTD	16	0.241870	4.6606 ± 0.3745	0.0744 ± 0.0168	0.0372 ± 0.0084	0.7359 ± 0.0000	0.7163 ± 0.0007
Classical BTD	29	0.041850	4.6624 ± 0.3404	0.0015 ± 0.0007	0.0007 ± 0.0003	0.8117 ± 0.0000	0.7419 ± 0.0094
BCA BTD	29	0.104965	4.6743 ± 0.2807	0.0267 ± 0.0207	0.0133 ± 0.0103	0.6272 ± 0.0000	0.7099 ± 0.0148

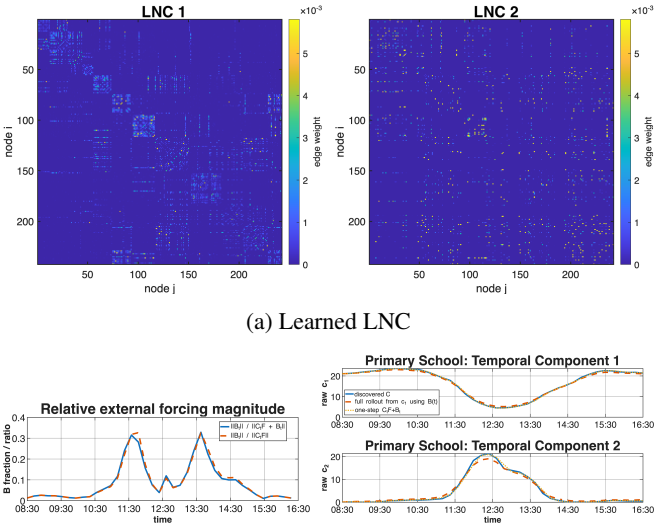
Table 2: Real-data reconstruction and LNC diagnostics using energy-rank based BTD block sizes.

Dataset	Energy	L_q	Method	Rel. Err.	Hoyer	Nuc. Norm	Cos. Dist.
SocioPatterns							
Primary School	–	–	DyLNC	**0.7957**	0.8488	9.3037	**0.8997**
Primary School	90%	38	Classical BTD	0.8075	0.8553	**7.9795**	0.8401
Primary School	90%	38	BCA-BTD	0.8088	0.8544	8.2817	0.8560
Primary School	95%	48	Classical BTD	0.8011	0.8549	8.3161	0.8395
Primary School	95%	48	BCA-BTD	0.8013	**0.8587**	8.6188	0.8565
S&P 500							
S&P 500	–	–	DyLNC	**0.6262**	0.5242	4.7097	**0.5990**
S&P 500	90%	10	Classical BTD	0.6353	0.4229	3.6472	0.3272
S&P 500	90%	10	BCA-BTD	0.6459	**0.5338**	3.9650	0.3883
S&P 500	95%	13	Classical BTD	0.6309	0.4393	**3.5615**	0.2762
S&P 500	95%	13	BCA-BTD	0.6387	0.5278	4.2149	0.3486

The S&P 500 results in Table 2 depict a similar behavior as the contact-network experiment. We choose $R = 4$ to balance between interpretability and evaluation metrics such that it gives separated, sector-interpretable LNC while keeping the coefficient dynamics low-dimensional. Increasing the BTD block rank improves the reconstruction of the tensor baselines, but the results remain sensitive to the prescribed L. In contrast, DyLNC obtains comparable or lower reconstruction error while maintaining larger cosine-distance separability than the tensor baselines. As shown in Fig. 3a, the learned latent LNCs capture distinct sector-sorted connectivity patterns, including different within-sector and cross-sector interaction structures. The corresponding coefficients in Fig. 3b are activated at different time periods, suggesting that the financial graph sequence evolves through a small number of latent market regimes. The peaks in the forcing fraction in Fig. 3c highlight intervals where additional exogenous input is required, and can be interpreted as data-driven signatures of large-scale market perturbations or regime changes. For instance, peaks in summer 2016 and early 2018 are in alignment with the Brexit referendum [19] and the "Volmageddon" volatility episode [20], respectively.

5. CONCLUSION AND FUTURE WORK

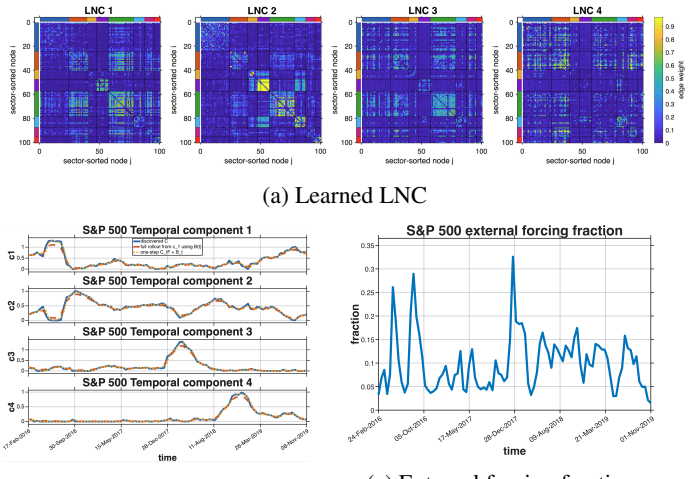
This paper presented DyLNC, a dynamics-aware latent network decomposition model for temporal networks. DyLNC represents each graph snapshot using distinct latent network components while jointly learning their activation coefficients, autonomous progression, and non-autonomous forcing effects. By combining graph regularization with multi-step dynamic consistency, the method provides interpretable structural components and a compact description of temporal



(a) Learned LNC

(b) External forcing fraction (c) Temporal components

Fig. 2: Primary school temporal-network decomposition results: learned LNC, external forcing contribution, and temporal components.



(a) Learned LNC

(b) Temporal components (c) External forcing fraction

Fig. 3: S&P 500: learned LNC, temporal progressions, and the relative contribution of external forcing.

evolution. Experiments on synthetic and real networks show that DyLNC recovers interpretable LNC, is less sensitive to BTD rank specification, and captures temporal progression beyond decomposition-only reconstruction.

Future work will extend the linear coefficient progression to nonlinear latent dynamics by lifting the coefficient state through nonlinear maps, enabling nonlinear dynamics analysis. We will also address the problem of selecting the number of latent components, R, in a principled way, such as using held-out reconstruction error, template separation, and consistency across random initializations.

6. REFERENCES

[1] Petter Holme and Jari Saramäki, "Temporal networks," *Physics Reports*, vol. 519, no. 3, pp. 97–125, 2012.

[2] Alain Barrat and Ciro Cattuto, "Temporal networks of face-to-face human interactions," in *Temporal Networks*, Understanding Complex Systems, pp. 191–216. Springer-Verlag Berlin Heidelberg, 2013.

[3] L. Zhao, G. J. Wang, M. Wang, W. Bao, W. Li, and H. E. Stanley, "Stock market as temporal network," *Physica A: Statistical Mechanics and its Applications*, vol. 506, pp. 1104–1112, 2018.

[4] D. M. Dunlavy, T. G. Kolda, and E. Acar, "Temporal link prediction using matrix and tensor factorizations," *ACM Transactions on Knowledge Discovery from Data*, vol. 5, no. 2, pp. 1–27, 2011.

[5] L. Gauvin, A. Panisson, and C. Cattuto, "Detecting the community structure and activity patterns of temporal networks: A non-negative tensor factorization approach," *PLoS ONE*, vol. 9, no. 1, pp. Art. no. e86028, Jan 2014.

[6] A. G. Mahyari and S. Aviyente, "Identification of dynamic functional brain network states through tensor decomposition," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Florence, Italy, 2014, pp. 2099–2103.

[7] A. G. Mahyari, D. M. Zoltowski, E. M. Bernat, and S. Aviyente, "A tensor decomposition-based approach for detecting dynamic network states from eeg," *IEEE Transactions on Biomedical Engineering*, vol. 64, no. 1, pp. 225–237, Jan 2017.

[8] E. Gujral, R. Pasricha, and E. E. Papalexakis, "Beyond rank-1: Discovering rich community structure in multi-aspect graphs," in *Proc. Web Conf. (WWW)*, Taipei, Taiwan, Apr 2020, pp. 452–462.

[9] Y. Shen, B. Baingana, and G. B. Giannakis, "Tensor decompositions for identifying directed graph topologies and tracking dynamic networks," *IEEE Transactions on Signal Processing*, vol. 65, no. 14, pp. 3675–3687, Jul 2017.

[10] B. Das, A. Buciulea, A. G. Marques, and E. Isufi, "Graph signal aware decomposition of dynamic networks via latent graphs," *arXiv preprint arXiv:2506.08519*, 2025.

[11] Z. Liu et al., "Graph odes and beyond: A comprehensive survey on integrating differential equations with graph neural networks," in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining (KDD)*, Toronto, ON, Canada, Aug 2025, pp. 6118–6128.

[12] Z. Huang, Y. Sun, and W. Wang, "Coupled graph ode for learning interacting system dynamics," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Mining (KDD)*, Virtual Event, Singapore, Aug 2021, pp. 705–715.

[13] Cesare Alippi and Daniele Zambon, "Graph kalman filters," *arXiv preprint arXiv:2303.12021*, 2023.

[14] Joshua L. Proctor, Steven L. Brunton, and J. Nathan Kutz, "Dynamic mode decomposition with control," *SIAM Journal on Applied Dynamical Systems*, vol. 15, no. 1, pp. 142–161, 2016.

[15] Y. Yang, M. Pesavento, Z.-Q. Luo, and B. Ottersten, "Inexact block coordinate descent algorithms for nonsmooth nonconvex optimization," *IEEE Transactions on Signal Processing*, vol. 68, pp. 947–961, 2020.

[16] Y. Zhu, J. Liu, and F. Cong, "Dynamic community detection for brain functional networks during music listening with block component analysis," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 31, pp. 2438–2447, 2023.

[17] Juliette Stehlé et al., "High-resolution measurements of face-to-face contact patterns in a primary school," *PLoS ONE*, vol. 6, no. 8, pp. e23176, 2011.

[18] A. Javaheri, J. Ying, D. P. Palomar, and F. Marvasti, "Time-varying graph learning for data with heavy-tailed distribution," *IEEE Transactions on Signal Processing*, vol. 73, pp. 3044–3060, 2025.

[19] A. Alkhatib and M. Harasheh, "Performance of exchange traded funds during the brexit referendum: An event study," *International Journal of Financial Studies*, vol. 6, no. 3, pp. 64, 2018.

[20] Patrick Augustin, Ing-Haw Cheng, and Ludovic Van den Bergen, "Volmageddon and the failure of short volatility products," *Financial Analysts Journal*, vol. 77, no. 3, pp. 35–51, 2021.